```python
import os
import sys
import pygame as pg
import random
import time


WIDTH, HEIGHT = 1100, 650
os.chdir(os.path.dirname(os.path.abspath(__file__)))


def check_bound(rct: pg.Rect) -> tuple[bool, bool]:
    """
    引数：こうかとんRectか、爆弾Rect
    戻り値：タプル（横方向結果判定，縦方向結果判定）
    画面内ならTrue，画面外ならFalse
    """
    yoko, tate = True, True
    if rct.left < 0 or WIDTH < rct.right:
        yoko = False
    if rct.top < 0 or HEIGHT < rct.bottom:
        tate = False
    return yoko, tate


def gameover(screen: pg.Surface) -> None:
    """
    引数：Screen
    戻り値：なし
    ゲームオーバー画面描写
    """
    black = pg.Surface((WIDTH, HEIGHT))
    pg.draw.rect(black, (0, 0, 0), pg.Rect(0, 0, WIDTH, HEIGHT))
    black.set_alpha(180)

    #文字実装
    fonto = pg.font.Font(None, 80)
    txt = fonto.render("Game Over", True, (255, 255, 255))
    txt_rct = txt.get_rect()
    txt_rct.center = WIDTH // 2, HEIGHT // 2
    black.blit(txt, txt_rct)

    # ｺｳｶﾄﾝ実装
    kk_img_l = pg.transform.rotozoom(pg.image.load("fig/8.png"), 0, 0.9)
    kk_img_r = pg.transform.rotozoom(pg.image.load("fig/8.png"), 0, 0.9)
    kk_rct_l = kk_img_l.get_rect()
    kk_rct_r = kk_img_r.get_rect()
    kk_rct_l.center = WIDTH // 2 - 200, HEIGHT // 2
    kk_rct_r.center = WIDTH // 2 + 200, HEIGHT // 2
    black.blit(kk_img_l, kk_rct_l)
    black.blit(kk_img_r, kk_rct_r)

    #画面描写
    screen.blit(black, [0, 0])
    pg.display.update()
    time.sleep(5)


def init_bb_imgs() -> tuple[list[pg.Surface], list[int]]:
    """
    引数：なし
```

```python
    戻り値：タプル（リスト（爆弾の画像），リスト（爆弾の速度））
    爆弾の大きさと速度を変えるためのタプルを出力
    """
    bb_imgs = []
    for r in range(1, 11): #大きさが異なる爆弾のリストを作成
        bb_img = pg.Surface((20*r, 20*r))
        pg.draw.circle(bb_img, (255, 0, 0), (10*r, 10*r), 10*r)
        bb_img.set_colorkey((0, 0, 0))
        bb_imgs.append(bb_img)
    bb_accs = [a for a in range(1, 11)] #爆弾の速度のリストを作成
    return bb_imgs, bb_accs


def get_kk_imgs() -> dict[tuple[int, int]: pg.Surface]:
    """
    引数：なし
    戻り値：辞書（タプル（x速度，y速度）：向きを変えたｺｳｶﾄﾝの画像）
    進む方向ごとにｺｳｶﾄﾝの向きを変えるための辞書を出力
    """
    kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
    kk_img_flip = pg.transform.flip(kk_img, True, False)
    kk_dict = {
        ( 0, 0): pg.transform.rotozoom(kk_img_flip, 0, 0.9), #何も押していない時
        ( 0,-5): pg.transform.rotozoom(kk_img_flip, 90, 0.9), #上
        (+5,-5): pg.transform.rotozoom(kk_img_flip, 45, 0.9), #右上
        (+5, 0): pg.transform.rotozoom(kk_img_flip, 0, 0.9), #右
        (+5, 5): pg.transform.rotozoom(kk_img_flip, -45, 0.9), #右下
        ( 0,+5): pg.transform.rotozoom(kk_img_flip, -90, 0.9), #下
        (-5,-5): pg.transform.rotozoom(kk_img, -45, 0.9), #左下
        (-5, 0): pg.transform.rotozoom(kk_img, 0, 0.9), #左
        (-5,+5): pg.transform.rotozoom(kk_img, 45, 0.9), #左上
    }
    return kk_dict


def show_life(life_num: int, screen: pg.Surface) -> None:
    """
    引数：int(残機), Screen
    戻り値：なし
    画面に残機を出力
    """
    kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
    screen.blit(kk_img, [0, 0])
    fonto = pg.font.Font(None, 80)
    txt = fonto.render(f" × {life_num}", True, (255, 255, 255))
    screen.blit(txt, [30, 0])



def main():
    pg.display.set_caption("逃げろ！こうかとん")
    screen = pg.display.set_mode((WIDTH, HEIGHT))

    #こうかとん初期化
    bg_img = pg.image.load("fig/pg_bg.jpg")
    kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
    kk_rct = kk_img.get_rect()
    kk_rct.center = 300, 200

    #爆弾初期化
    bb_img = pg.Surface((20, 20))
    pg.draw.circle(bb_img, (255, 0, 0), (10, 10), 10)
    bb_img.set_colorkey((0, 0, 0))
    bb_rct = bb_img.get_rect()
    bb_rct.center = random.randint(0, 1100), random.randint(0, 650)
    vx, vy = +5, +5

    clock = pg.time.Clock()
    tmr = 0
    DELTA = {
        pg.K_UP:(0, -5),
        pg.K_DOWN:(0, +5),
        pg.K_LEFT:(-5, 0),
        pg.K_RIGHT:(+5, 0),
    }
    bb_imgs, bb_accs = init_bb_imgs()
    kk_imgs = get_kk_imgs()
```

```python
    life = 3
    alive = True
    while True:
        for event in pg.event.get():
            if event.type == pg.QUIT:
                return
        if kk_rct.colliderect(bb_rct) and alive:
            alive = False
            life -= 1
            if life == -1:
                print("ゲームオーバー")
                gameover(screen)
                return
        elif not kk_rct.colliderect(bb_rct):
            alive = True
        screen.blit(bg_img, [0, 0])

        show_life(life, screen)

        key_lst = pg.key.get_pressed()
        sum_mv = [0, 0]
        for key, mv in DELTA.items():
            if key_lst[key]:
                sum_mv[0] += mv[0] #横方向の移動
                sum_mv[1] += mv[1] #縦方向の移動
        kk_rct.move_ip(sum_mv)
        if check_bound(kk_rct) != (True, True):
            kk_rct.move_ip(-sum_mv[0], -sum_mv[1]) #動きをキャンセル
        kk_img = kk_imgs[tuple(sum_mv)]
        screen.blit(kk_img, kk_rct)

        avx = vx * bb_accs[min(tmr // 500, 9)]
        avy = vy * bb_accs[min(tmr // 500, 9)]
        bb_img = bb_imgs[min(tmr // 500, 9)]
        bb_rct.width = bb_img.get_rect().width
        bb_rct.height = bb_img.get_rect().height
        bb_rct.move_ip([avx, avy])
        yoko, tate = check_bound(bb_rct)
        if not yoko:
            vx *= -1
        if not tate:
            vy *= -1
        screen.blit(bb_img, bb_rct)
        pg.display.update()
        tmr += 1
        clock.tick(50)


if __name__ == "__main__":
    pg.init()
    main()
    pg.quit()
    sys.exit()
```